

BÁO CÁO QUÁ TRÌNH THỰC HIỆN ĐỀ TÀI
CHUYÊN ĐỀ: ĐIỀU KHIỂN ĐỘNG CƠ BẰNG
PHƯƠNG PHÁP PID KẾT HỢP ANN

Người thực hiện: MSSV:

Nguyễn Anh Hào 2314015

GVHD:

Cái Việt Anh Dũng

SVHD:

Nguyễn Đình Anh Khôi

Hồ Chí Minh, ngày 20 tháng 11 năm 2025

TABLE OF CONTENT

TÌM HIỂU ĐỘNG CƠ NF5475 VÀ ESP32 WROOM.....	2
Động cơ NF5475.....	2
ESP32 WROOM.....	4
GIAO DIỆN LẬP TRÌNH ADRUINO IDLE.....	5
PID Controller.....	6
ĐIỀU KHIỂN ĐỘNG CƠ.....	9
Khai báo hàm.....	9
QUY TRÌNH TÍNH TOÁN THÔNG SỐ PI VẬN TỐC.....	12
Xác định hàm đặc tính của động cơ.....	12
Xác định thông số K_p, K_i.....	12
QUY TRÌNH TRAIN MẠNG ANN DỰ ĐOÁN K_p, K_i.....	14
Chuẩn bị data.....	14
Các hàm khởi tạo.....	14
- Hàm activation.....	14
- Hàm Dense:.....	16
- Hàm loss:.....	17
- Hàm optimizer:.....	18
Chuẩn hóa dữ liệu.....	19
Tạo mô hình.....	20
Train mô hình.....	20
Đánh giá hiệu quả mô hình.....	21

TÌM HIỂU ĐỘNG CƠ NF5475 VÀ ESP32 WROOM

Động cơ NF5475

Động cơ NF5475 là động cơ DC Brushed có thêm cơ cấu Encoder gắn liền phía sau động cơ.

Điện áp: 12-24V (5V Encoder)

Công suất: 33W

Đường kính trục: 8mm

Độ phân giải xung: 200 PPR

TỔNG QUAN SẢN PHẨM ĐỘNG CƠ NISCA NF5475

Động cơ NF5475F NISCA DC servo motor là loại động cơ DC Brushed(Động cơ chổi than DC) có thêm cơ cấu Encoder gắn liền phía sau của động cơ. Đây là loại động cơ khá thông dụng, đa phần được sử dụng là loại được tháo từ máy Photocopy. Tuy nhiên nguồn gốc hàng hóa cũng như chất lượng của sản phẩm này được đánh giá rất tốt và có mức giá rẻ phù hợp với phần lớn đối tượng sử dụng.



Hình ảnh động cơ NISCA NF5475

NISCA Corporation là 1 thương hiệu lớn của Nhật Bản về động cơ, thương hiệu này chuyên cung cấp các loại động cơ DC Servo, BLDC sử dụng trong các máy in, máy photocopy của các thương hiệu lớn như Cannon, Epson, Brother, Ricoh..... Do đó, dù là hàng tháo máy tuy nhiên chất lượng của động cơ và bộ encoder của động cơ là rất tốt, ví dụ như:

- Độ ổn định cao
- Độ nhiễu gây ra bởi động cơ lên mạch điều khiển thấp
- Tiếng ồn nhỏ.
- Chất lượng xung Encoder A, B hoàn hảo
- Motor không phát nóng nhiều trong quá trình sử dụng/;

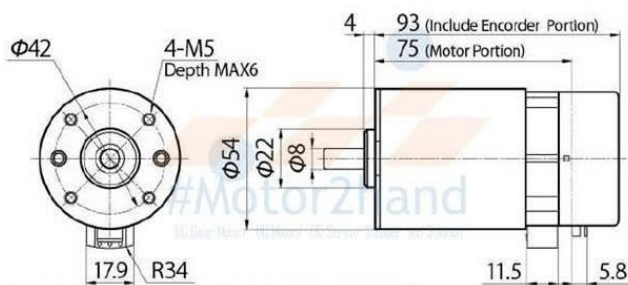
THÔNG SỐ KỸ THUẬT ĐỘNG CƠ

Model No.	at Maximum Efficiency					at Continuous Operation(at 50°C)	No Load		Starting	
	Torque [mN · m]	Speed [rpm]	Current [A]	Output [W]	Efficiency [%]	Possible Max Output [W]	Speed [rpm]	Current [A]	Torque [mN · m]	Current [A]
NAS475F	71.07	3234	1.494	24.07	67.1	33.7	3818	0.270	464.10	8.264
NF5475A	81.40	3199	1.617	27.27	70.3	38.9	3716	0.261	585.10	10.003
NF5475E	74.83	4162	1.256	32.62	68.3	38.9	4884	0.218	506.57	7.246

- Điện áp: 12-24V
- Công suất: 33W
- Dòng tải: 1.49 A
- Dòng không tải: 0.27A
- Dòng khởi động: 8.264A
- Momen khởi động: 464mN*m
- Momen tải: 71.07mN*m
- Tốc độ không tải: 3818 RPM

THÔNG SỐ KÍCH THƯỚC ĐỘNG CƠ

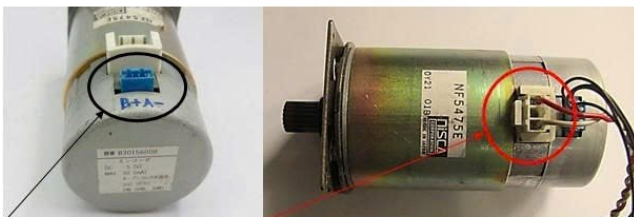
- Đường kính trục động cơ: 8mm
 - Đường kính thân động cơ: 54mm
 - Chiều dài động cơ không tính trục (bao gồm cả encoder): 93mm
 - Chiều dài không bao gồm phần encoder: 75mm
 - Lỗ mặt bích động cơ: Lỗ ren M5
- Chi tiết xem tại hình sau:



THÔNG SỐ ENCODER

Encoder Characteristic						
Output Phase	Resolution [P/R]	Supply Voltage [V]	Current [mA]	Frequency Response [kHz]	Encoder Type	Signal Format
2	100	5	MAX 50	10	Optical	Incremental
2	200	5	MAX 50	20	Optical	Incremental

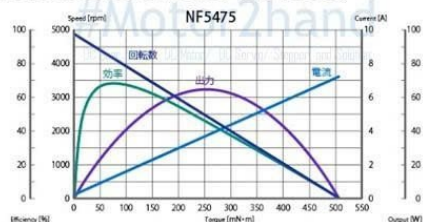
- Điện áp: 5VDC
- Số lượng dây: 4 dây (4 wires)
- Số lượng kênh: 2 kênh A, B (A, B pulse channel)
- Phân dải xung: 200 PPR (xung/ vòng)
- Tốc độ phản hồi xung: 20 KHz



Bố trí dây Encoder
(+) +5V Vcc (A) Kênh xung A
(-) 0V GND (B) Kênh xung B

Bố trí dây nguồn cấp động cơ
(Dây đỏ) VDD 12V hoặc 24V
(Dây trắng) GND 0V

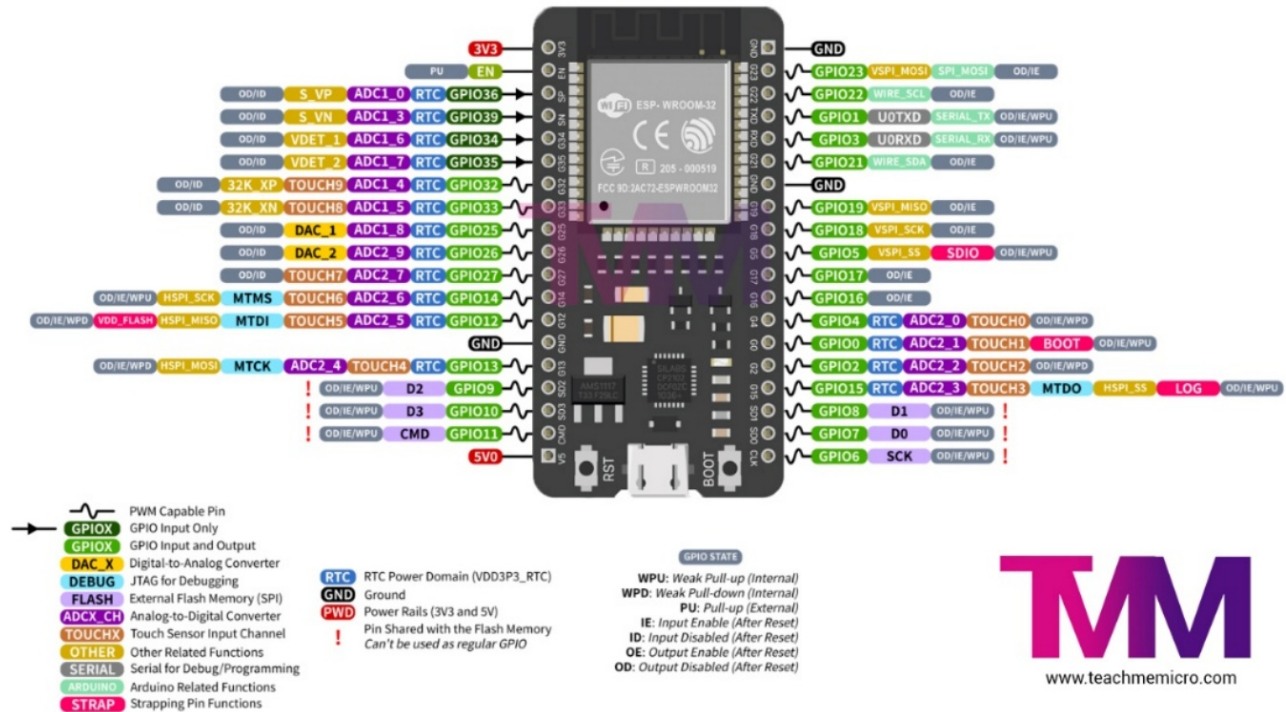
BIỂU ĐỒ ĐẶC TÍNH MOMEN, DÒNG TẢI, TỐC ĐỘ



ESP32 WROOM

ESP32 PINOUT DIAGRAM

ESP32-WROOM-32



Cấu hình chân sử dụng:

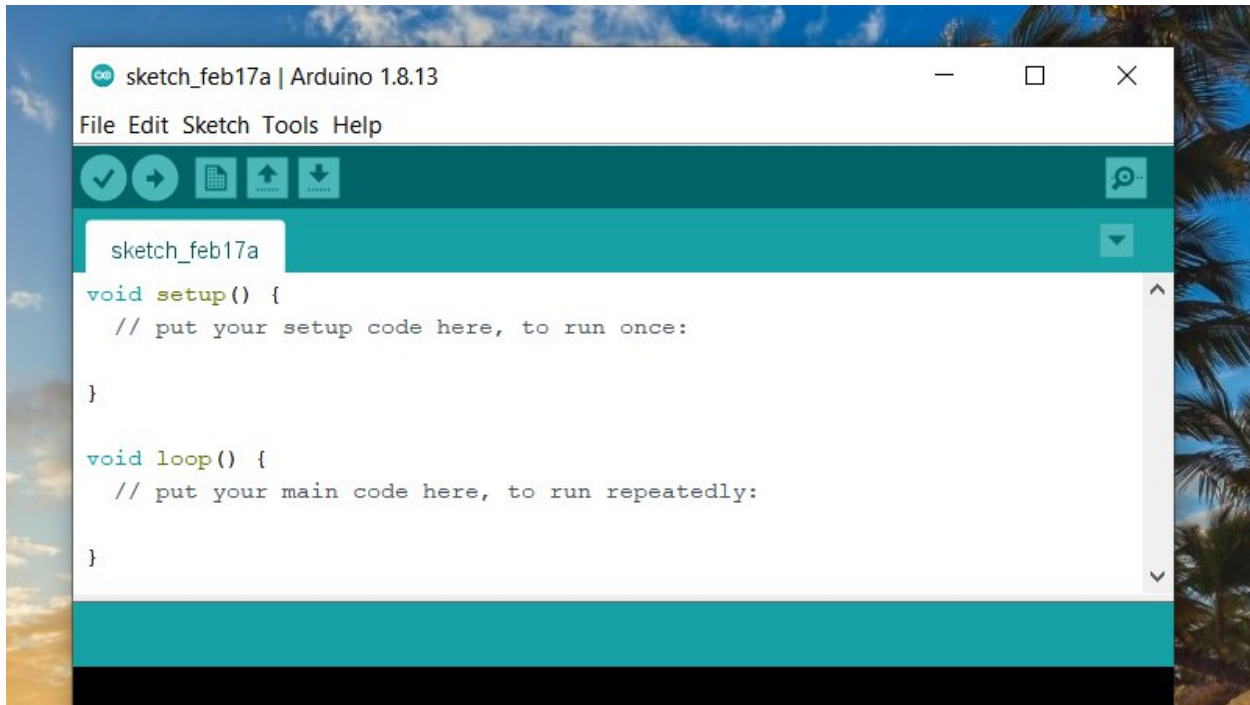
18, 19, 25, 26: GPIO

5V0: cấp nguồn 5V

GND: nối đất

GIAO DIỆN LẬP TRÌNH ARDUINO IDE

Arduino IDE là môi trường phát triển chính thức dùng để lập trình các bo mạch Arduino như Uno, Nano, Mega, Leonardo, ESP8266/ESP32 (qua cài đặt thêm), v.v.



Void setup(){}: mã lệnh ở đây được thực thi 1 lần duy nhất, nhằm mục đích khởi tạo các giá trị tham số.

Void loop(){}: mã lệnh ở đây được thực thi tuần hoàn, đây là chương trình chính, chúng ta sẽ xử lý các tác vụ ở đây.

PID Controller

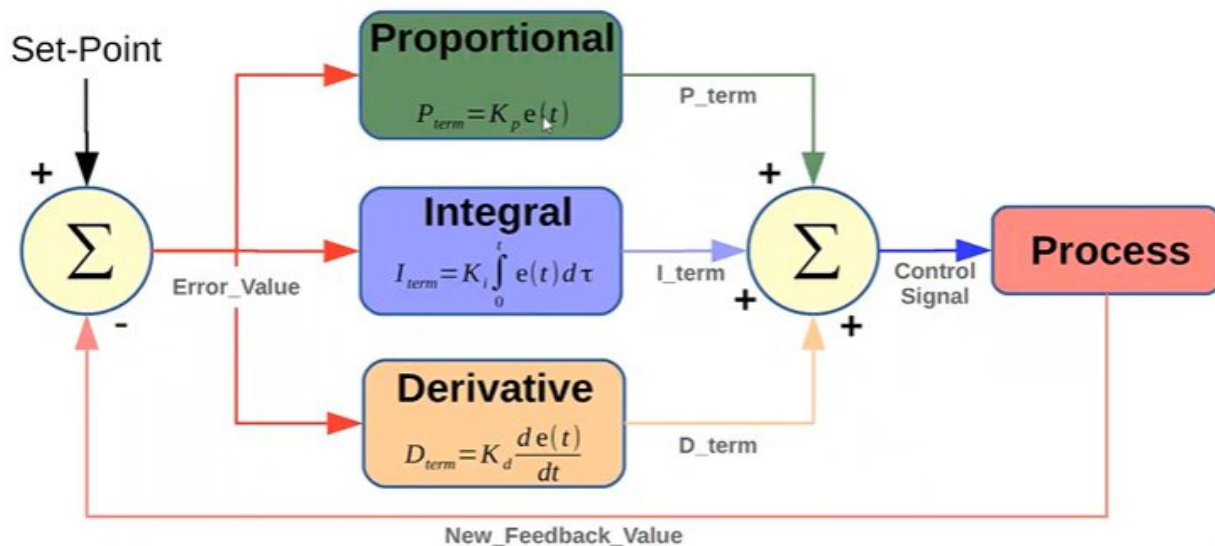
PID Controller: Proportional (P), Integral (I), Derivative (D) Controller.

Là bộ điều khiển vi tích phân tỉ lệ vòng kín có phản hồi. Dùng để điều khiển ngõ ra (output) giống với tín hiệu đặt (input - setpoint).

PID làm ổn định hệ thống điều khiển: loại bỏ sai số xác lập, đáp ứng nhanh và giảm vọt lố khi tìm được K_p , K_i , K_d phù hợp.

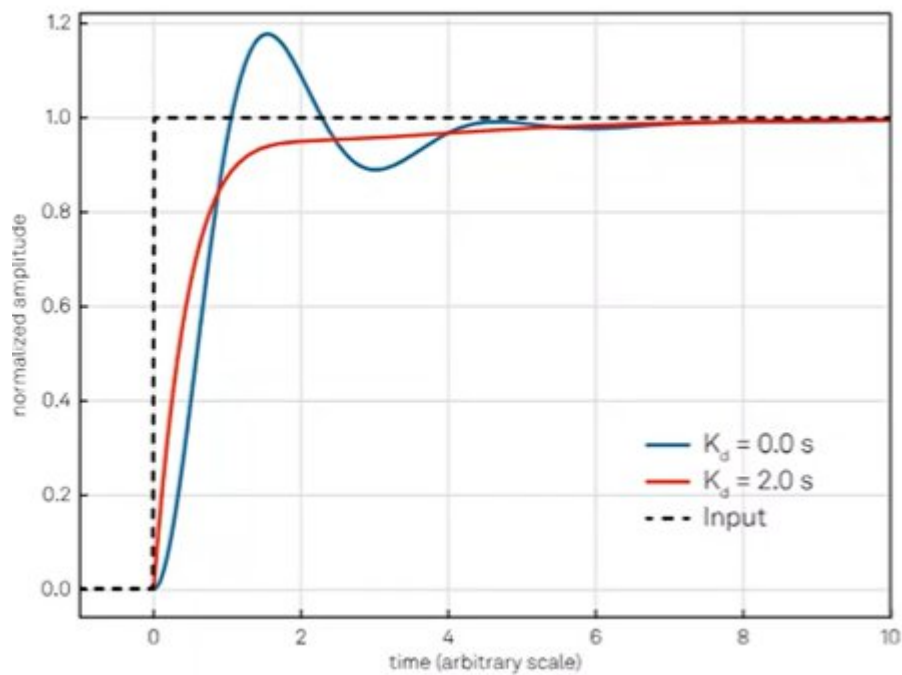
Công thức tổng quát:

$$u(t) = u_P(t) + u_I(t) + u_D(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{d}{dt} e(t)$$



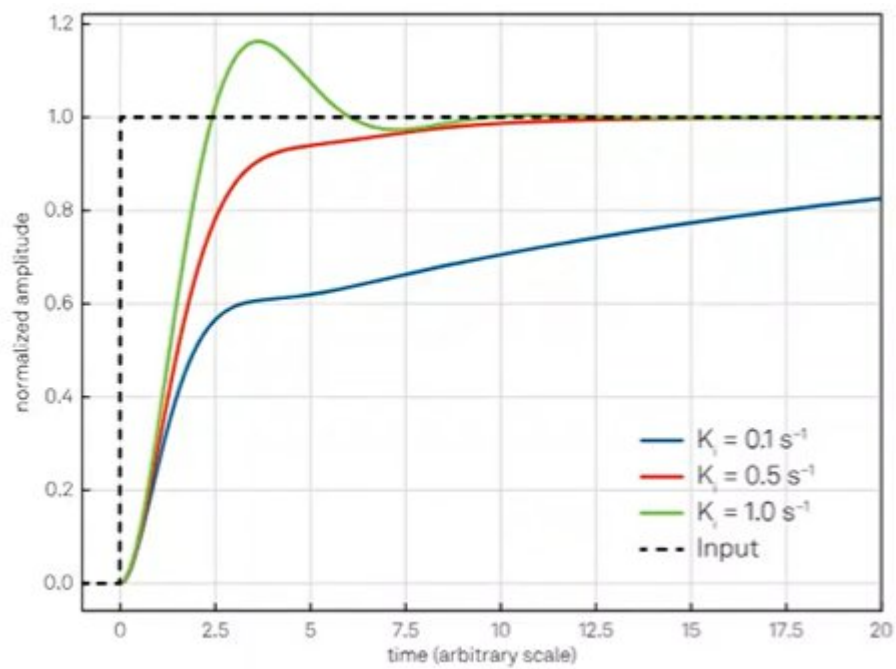
Khâu P:

K_p càng lớn đáp ứng càng nhanh, sai số xác lập càng nhỏ, tăng độ vọt lố, mất ổn định à tăng thời gian xác lập, gây vọt lố.



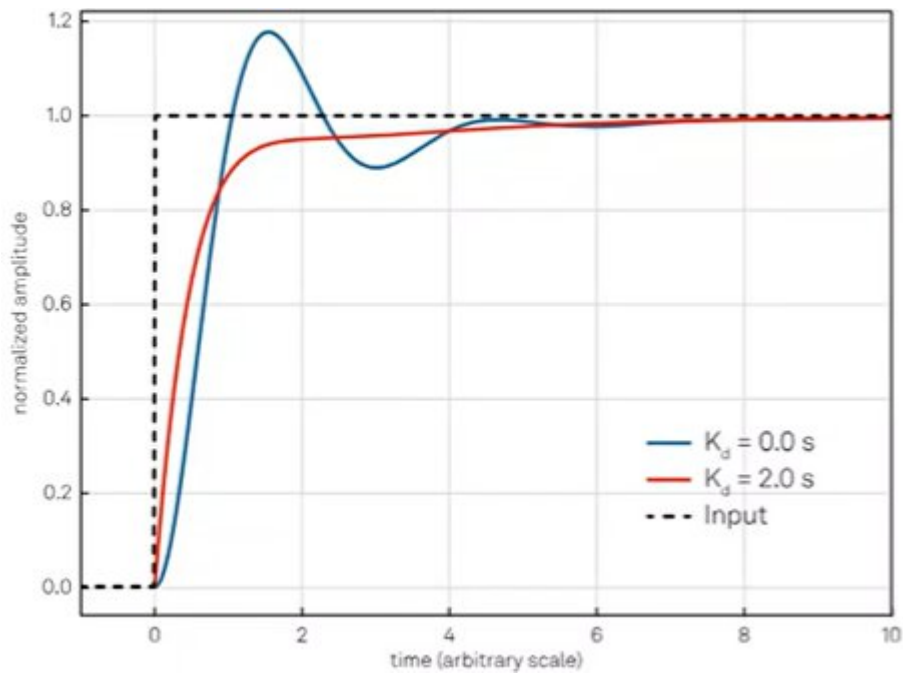
Khâu I:

Đưa hệ thống gần giá trị đích và không để lại sai số, K_i càng lớn độ vọt lố càng cao, sai số xác lập càng nhỏ, đáp ứng quá độ chậm ả loại bỏ dao động, tăng thời gian xác lập.

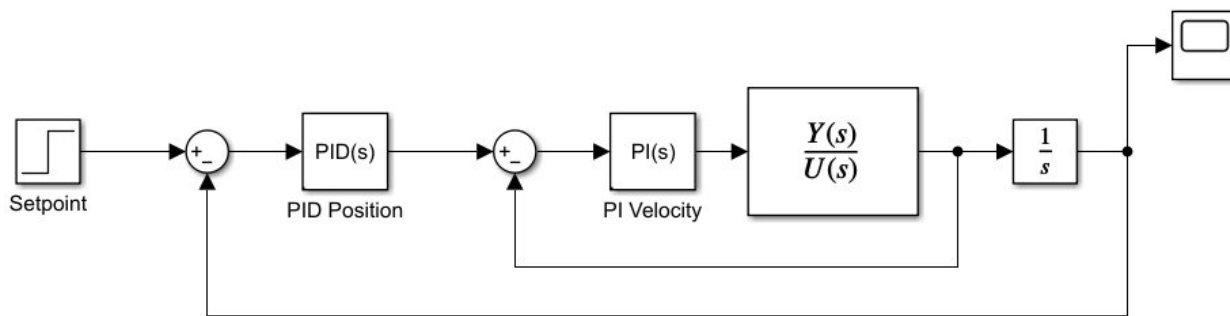


Khâu D:

Làm chậm hệ thống khi tiếp cận giá trị đích, giảm quá độ và dao động, làm hệ thống ổn định nhanh, K_d càng lớn đáp ứng quá độ càng nhanh, độ vọt lố nhỏ à Giảm quá độ và thời gian ổn định.



Áp dụng vào đề tài, mô hình động cơ điều khiển:



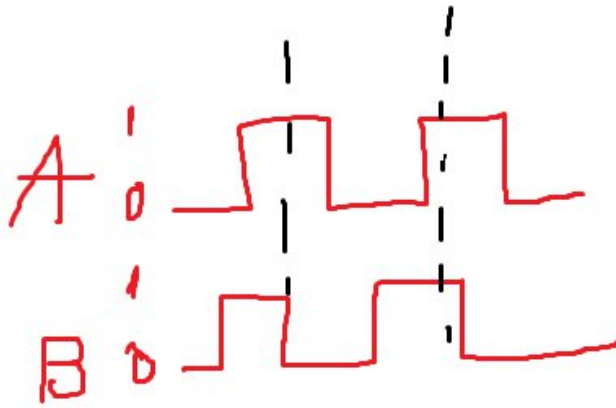
ĐIỀU KHIỂN ĐỘNG CƠ

Khai báo hàm

Hàm ngắt encoder:

```
void IRAM_ATTR encoderISR() {  
    if (digitalRead(phase_a) == digitalRead(phase_b))  
        pulse--;  
    else  
        pulse++;  
}
```

Chức năng: đếm xung, nhằm xác định vị trí, chiều quay hoặc tính vận tốc quay. Hàm được kích hoạt khi có tín hiệu ngắt ISR, lúc này ta xét điều kiện phase_a và phase_b. Khi quay thuận thì phase_a sớm pha hơn phase_b, ngược lại phase_b sớm pha hơn phase_a khi quay nghịch. Vị trí và chiều quay.



Hàm tính vận tốc:

```
float VelCalculate(float Pos_now, float delta) {  
    float Current_vel = (Pos_now / 200) / delta * 60;  
    return Current_vel;  
}
```

Chức năng: trả về giá trị float số vòng quay trong 1 phút (RPM), tối đa là 3100 RPM.

Khởi tạo:

```

void setup() {
  Serial.begin(115200);
  pinMode(R_PWM, OUTPUT);
  pinMode(L_PWM, OUTPUT);
  pinMode(phase_a, INPUT_PULLUP);
  pinMode(phase_b, INPUT_PULLUP);
  attachInterrupt(phase_a, encoderISR, CHANGE);

  ledcAttach(R_PWM, 20000, 8);
  ledcAttach(L_PWM, 20000, 8);
  ledcWrite(R_PWM, 0);
  ledcWrite(L_PWM, 0);
}

```

Cài đặt chế độ hoạt động của chân xuất PWM và chân tín hiệu encoder, ở đây là PWM 8bit max [-255, 255]. Phase_a là chân interrupt hoạt động chế độ 1x (chỉ xét trên 1 cạnh của xung A.

Điều khiển chiều quay:

```

// PWM xuất ra động cơ
if (pwm > 0) {
  ledcWrite(R_PWM, pwm);
  ledcWrite(L_PWM, 0);
} else {
  ledcWrite(R_PWM, 0);
  ledcWrite(L_PWM, -pwm);
}

```

Để động cơ có thể quay ta cấp vào xung PWM tương ứng với chiều muốn quay.

In\truyền dữ liệu ra cổng serial:

```

Serial.print(t, 3); Serial.print(",");
//Serial.print(pos_now); Serial.print(",");
Serial.print(current_vel); Serial.print(",");
Serial.print(velocityReq); Serial.print(",");
//Serial.print(output); Serial.print(",");
//Serial.print(error_pos); Serial.print(",");
Serial.println(error_vel);

```

Chức năng: hiển thị dữ liệu tính toán sau mỗi $\Delta t = 0.01s$, ở đây gồm:

- t (thời gian): xác định thời gian chạy
- $current_vel$ (vận tốc hiện tại): được trả về từ hàm tính vận tốc
- $velocityReq(setpoint\ vận\ tốc)$: vận tốc cần đạt
- $error_vel$ (sai số vận tốc): hiệu chênh lệch giữa setpoint vận tốc và vận tốc hiện tại

PI vận tốc:

```
current_vel = velCalculate(pos_vel_cal, Delta);
pos_vel_cal = 0;
error_vel = velocityReq - current_vel;
integral_vel += error_vel * Delta;
integral_vel = constrain(integral_vel, -1000, 1000);
float derivative_vel = (error_vel - error_vel_prev) / Delta;

output = int(kp_vel * error_vel + ki_vel * integral_vel + kd_vel * derivative_vel);

output = constrain(output, -255, 255);
pwm = output;
```

Dựa trên công thức PID, tính ra error, tích phân error và đạo hàm error ($K_d = 0$). Có anti-windup khâu I, giới hạn lại PWM.

QUY TRÌNH TÍNH TOÁN THÔNG SỐ PI VẬN TỐC

Xác định hàm đặc tính của động cơ

Hàm đặc tính của động cơ được xác định thông qua việc thu data (time, vel, pwm) mỗi 20% của max PWM trong 10s, sau đó đưa vào công cụ tfest matlab để giả lập mô hình trạng thái động cơ. Dựa trên đáp ứng vận tốc tại các mốc PWM, matlab xác định được mô hình động cơ là:

$$G(s) = \frac{353.5s^3 + 81.55s^2 + 16.27s + 3.984}{s^4 + 33s^3 + 7.444s^2 + 3489s + 0.2802}$$

Với đánh giá chính xác so với data thực là: 95.36%, FPE: 2572, MSE: 2562

Xác định thông số Kp, Ki

Sau khi có được G(s) kết hợp với ràng buộc về khoảng giới hạn Kp, Ki, PWM tối đa, anti-windup I, người thực hiện đã thử 3 phương pháp tính toán thông số PI:

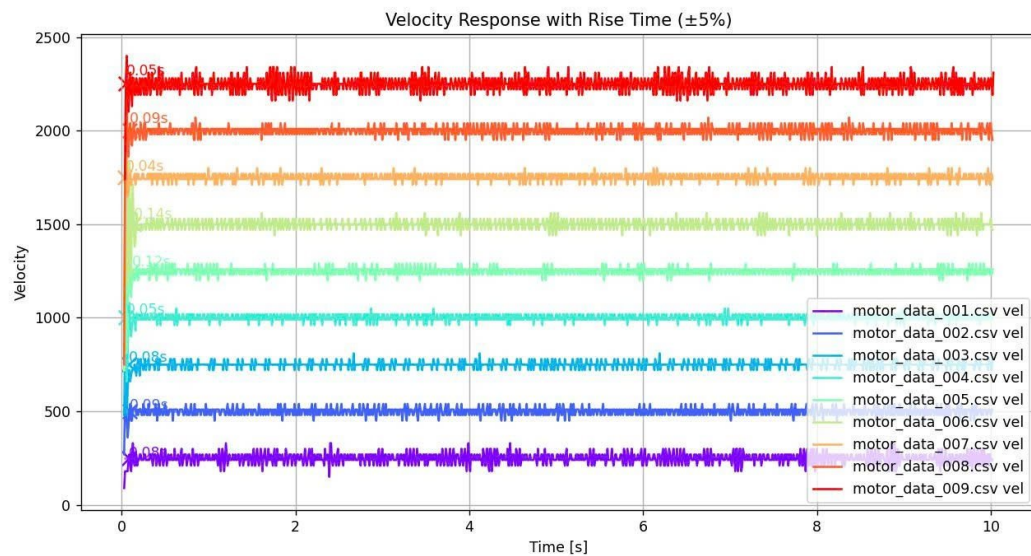
- Random gain in bounds (500 epochs each time)
- Tuner tool in matlab
- GA (Genetic Algorithims)

Trong đó phương pháp GA mang lại kết quả tốt nhất khi các bộ gain thu được có độ vọt lố (overshot) < 5%, rise-time < 0.5s, fit-to-data ~95%.

Trải qua các quá trình trên, người thực hiện thu về 9 bộ data (time, vel, error, delta_error, setpoint_vel) từ setpoint 250 đến setpoint 2250, mỗi bộ gồm 1001 dòng (10s).

	A	B	C	D	E	F	G
1	time	vel	error_vel	delta_error	velocityReq		
2	0.03	90	160	0	250		
3	0.04	180	70	-90	250		
4	0.05	180	70	0	250		
5	0.06	180	70	0	250		
6	0.07	180	70	0	250		
7	0.08	240	10	-60	250		
8	0.09	270	-20	-30	250		
9	0.1	210	40	60	250		
10	0.11	180	70	30	250		
11	0.12	240	10	-60	250		
12	0.13	330	-80	-90	250		
13	0.14	240	10	90	250		
14	0.15	210	40	30	250		
15	0.16	270	-20	-60	250		
16	0.17	270	-20	0	250		
17	0.18	210	40	60	250		
18	0.19	240	10	-30	250		
19	0.2	240	10	0	250		
20	0.21	270	-20	-30	250		
21	0.22	270	-20	0	250		
22	0.23	240	10	30	250		
23	0.24	240	10	0	250		
24	0.25	240	10	0	250		
25	0.26	270	-20	-30	250		
26	0.27	270	-20	0	250		

A	B	C	D	E
bestKp	bestKi	vel_setpoint		
0.185436	5.288458	250		



QUY TRÌNH TRAIN MẠNG ANN DỰ ĐOÁN KP, KI

Chuẩn bị data

9 bộ data được chia thành 7 bộ train (77.7%) và 2 bộ test (22.3%), được gộp chung thành 1 file train và 1 file test.

Data_Train > merger_vel > merged_train.csv > data									
1	time	vel	error_vel	delta_error_vel	velocityReq	gain_bestKp	gain_bestKi		
6984	9.85	1770.0	-20.0	0.0	1750.0	0.225015260695195	6.26604780858816		
6985	9.86	1740.0	10.0	30.0	1750.0	0.225015260695195	6.26604780858816		
6986	9.87	1710.0	40.0	30.0	1750.0	0.225015260695195	6.26604780858816		
6987	9.88	1770.0	-20.0	-60.0	1750.0	0.225015260695195	6.26604780858816		
6988	9.89	1770.0	-20.0	0.0	1750.0	0.225015260695195	6.26604780858816		
6989	9.9	1740.0	10.0	30.0	1750.0	0.225015260695195	6.26604780858816		
6990	9.91	1740.0	10.0	0.0	1750.0	0.225015260695195	6.26604780858816		
6991	9.92	1740.0	10.0	0.0	1750.0	0.225015260695195	6.26604780858816		
6992	9.93	1770.0	-20.0	-30.0	1750.0	0.225015260695195	6.26604780858816		
6993	9.94	1770.0	-20.0	0.0	1750.0	0.225015260695195	6.26604780858816		
6994	9.95	1710.0	40.0	60.0	1750.0	0.225015260695195	6.26604780858816		
6995	9.96	1740.0	10.0	-30.0	1750.0	0.225015260695195	6.26604780858816		
6996	9.97	1800.0	-50.0	-60.0	1750.0	0.225015260695195	6.26604780858816		
6997	9.98	1740.0	10.0	60.0	1750.0	0.225015260695195	6.26604780858816		
6998	9.99	1710.0	40.0	30.0	1750.0	0.225015260695195	6.26604780858816		
6999	10.0	1770.0	-20.0	-60.0	1750.0	0.225015260695195	6.26604780858816		
7000	10.01	1770.0	-20.0	0.0	1750.0	0.225015260695195	6.26604780858816		
7001	10.02	1740.0	10.0	30.0	1750.0	0.225015260695195	6.26604780858816		
7002									

Ngoài ra còn tạo thêm 1 bộ validation được copy từ 15% bộ train và đã được xáo trộn nhằm giúp tinh chỉnh thông số mô hình.

Các hàm khởi tạo

- Hàm activation

```

class Activation_ReLU:
    # Calculate the prediction for the output
    def predictions(self, outputs):
        return outputs

    #Forward Pass
    def forward(self,inputs):
        #Calculate output values from input
        self.inputs = inputs
        self.output = np.maximum(0,inputs)

    # Backward Pass
    def backward(self,dvalues):
        self.dinputs = dvalues.copy()
        # Put to zero gradient where input is negative
        self.dinputs[self.inputs <= 0] = 0

```

Hiệu đơn giản thì các giá trị âm được thay thế = 0, các giá trị > 0 giữ nguyên.

```

class Activation_Linear:

    # Calculate the prediction for the output
    def predictions(self, outputs):
        return outputs

    # Forward Pass
    def forward(self,inputs):
        #Calculate output values from input
        self.inputs = inputs
        self.output = inputs

    # Backward Pass
    def backward(self,dvalues):
        self.dinputs = dvalues.copy()

```

Các giá trị chỉ được forward qua, không thay đổi.

Ở đây sử dụng 2 hàm activation này nhằm phục vụ bài toán hồi quy (Regression).

- **Hàm Dense:**

```

class Layer_Dense_Regularization:
    def __init__(self, n_inputs, n_neurons, weights_regularizer_l1 = 0, weights_regularizer_l2 = 0, biases_regularizer_l1 = 0, biases_regularizer_l2 = 0):
        # Init weights and biases
        self.weights = 0.01*np.random.randn(n_inputs,n_neurons)
        self.biases = np.zeros((1,n_neurons))
        # Set regularization strength
        self.weights_regularizer_l1 = weights_regularizer_l1
        self.weights_regularizer_l2 = weights_regularizer_l2
        self.biases_regularizer_l1 = biases_regularizer_l1
        self.biases_regularizer_l2 = biases_regularizer_l2

    # forward pass
    def forward(self,inputs):
        #calculate outputs
        self.output = np.dot(inputs,self.weights) + self.biases
        self.inputs = inputs

    # backward pass
    def backward(self,dvalues):
        # Gradients on parameters
        self.dweights = np.dot(self.inputs.T, dvalues)
        self.dbiases = np.sum(dvalues, axis = 0, keepdims = True)

        # Gradients on regularization
        # L1 on weights
        if (self.weights_regularizer_l1 > 0):
            dl1 = np.ones_like(self.weights) # return an array filled of 1, same shape as weights array
            dl1[self.weights < 0] = -1
            self.dweights += self.weights_regularizer_l1 * dl1

        # L2 on weights
        if (self.weights_regularizer_l2 > 0):
            self.dweights += 2 * self.weights_regularizer_l2 * self.weights

        # L2 on biases
        if (self.biases_regularizer_l2 > 0):
            self.dbiases += 2 * self.biases_regularizer_l2 * self.biases

        # Gradients on values
        self.dinputs = np.dot(dvalues,self.weights.T)

    def get_parameters(self):
        return self.weights,self.biases

    def set_parameters(self,weights,biases):
        self.weights = weights
        self.biases = biases

```

Đây là lớp có trọng số regularizer nhằm chống overfitting bằng cách phạt (thêm loss) để đơn giản hóa mô hình khi variance cao.

Ngược lại là underfitting khi biases cao.

```

class Dense_Dropout:
    def __init__(self, rate):
        self.rate = 1 - rate # invert the rate

    # forward pass
    def forward(self,inputs):
        self.inputs = inputs

        # generate a scaled mask
        self.binary_mask = np.random.binomial(1, self.rate, size = inputs.shape) / self.rate

        # Compute the output value
        self.output = inputs * self.binary_mask

    # backward pass
    def backward(self,dvalues):
        self.dinputs = dvalues * self.binary_mask

```

Loại bỏ đi 1 vài trọng số giúp mô hình tìm được các điểm cực tiểu tốt hơn và giảm overfitting.

- Hàm loss:

```
class Loss_MeanSquaredError(Loss):

    # Forward Pass
    def forward(self, y_pred, y_true):
        # Calculate loss
        sample_losses = np.mean((y_true - y_pred)**2, axis = -1) # axis = -1 represents the last axis
        return sample_losses

    # Backward Pass
    def backward(self, dvalues, y_true):
        # Determine the number of samples
        samples = len(dvalues)

        # number of outputs in every sample, here we use the first sample to count
        outputs = len(dvalues[0])

        # Calculate gradient
        self.dinputs = -2 * (y_true - dvalues) / outputs
        #dx = dx / samples
        self.dinputs = np.array([dx, np.zeros(len(dx))]).T
        #self.dinputs = np.array([dx, dx]).T

        # Normalize gradient
        self.dinputs = self.dinputs / samples

    # Regularization
    def regularization_loss(self):
        # set regularization to 0 by default
        regularization_loss = 0

        for layer in self.trainable_layers:
            # Calculate L1 regularization for weights
            if (layer.weights_regularizer_l1 > 0):
                regularization_loss += layer.weights_regularizer_l1 * np.sum(np.abs(layer.weights))

            # Calculate L2 regularization for weights
            if (layer.weights_regularizer_l2 > 0):
                regularization_loss += layer.weights_regularizer_l2 * np.sum(layer.weights**2)

            # Calculate L1 regularization for biases
            if (layer.biases_regularizer_l1 > 0):
                regularization_loss += layer.biases_regularizer_l1 * np.sum(np.abs(layer.biases))

            # Calculate L2 regularization for biases
            if (layer.biases_regularizer_l2 > 0):
                regularization_loss += layer.biases_regularizer_l2 * np.sum(layer.biases**2)

        return regularization_loss
```

MSE là hàm loss thường được dùng cho các bài toán hồi quy.

- Hàm optimizer:

```

class Optimizer_Adam:

    def __init__(self, learning_rate = 0.1, decay = 5e-3, epsilon = 1e-7, beta1 = 0.9, beta2 = 0.9):
        self.learning_rate = learning_rate
        self.current_learning_rate = learning_rate
        self.decay = decay
        self.step = 0
        self.epsilon = epsilon
        self.beta1 = beta1
        self.beta2 = beta2

    # pre update
    def pre_update_params(self):
        if self.decay:
            self.current_learning_rate = self.learning_rate * (1 / (1 + self.decay * self.step))

    # Update parameters
    def update_params(self, layer):

        if not hasattr(layer, 'weights_cache'):
            # if layers do not contain momentum, create them then fill with zeros
            layer.weights_momentum = np.zeros_like(layer.weights)
            layer.weights_cache = np.zeros_like(layer.weights)
            layer.biases_momentum = np.zeros_like(layer.biases)
            layer.biases_cache = np.zeros_like(layer.biases)

        # Update momentum with current gradient
        layer.weights_momentum = self.beta1 * layer.weights_momentum + (1 - self.beta1) * layer.dweights
        layer.biases_momentum = self.beta1 * layer.biases_momentum + (1 - self.beta1) * layer.dbiases

        # Update momentum with current gradient
        layer.weights_momentum = self.beta1 * layer.weights_momentum + (1 - self.beta1) * layer.dweights
        layer.biases_momentum = self.beta1 * layer.biases_momentum + (1 - self.beta1) * layer.dbiases

        # Correct the momentum. step must start with 1 here
        weights_momentum_corrected = layer.weights_momentum / (1 - self.beta1**(self.step + 1))
        biases_momentum_corrected = layer.biases_momentum / (1 - self.beta1**(self.step + 1))

        # update cache
        layer.weights_cache = self.beta2 * layer.weights_cache + (1 - self.beta2) * layer.dweights**2
        layer.biases_cache = self.beta2 * layer.biases_cache + (1 - self.beta2) * layer.dbiases**2

        # Obtain the corrected cache
        weights_cache_corrected = layer.weights_cache / (1 - self.beta2**(self.step + 1))
        biases_cache_corrected = layer.biases_cache / (1 - self.beta2**(self.step + 1))

        # Update weights and biases
        layer.weights += -self.current_learning_rate * weights_momentum_corrected / (np.sqrt(weights_cache_corrected) + self.epsilon)
        layer.biases += -self.current_learning_rate * biases_momentum_corrected / (np.sqrt(biases_cache_corrected) + self.epsilon)

    # post update
    def post_update_params(self):
        self.step += 1

```

Có thể dùng các thuật toán khác, ở đây người thực hiện dùng thuật toán Adam.

Chuẩn hóa dữ liệu

```
EPS = 1e-8
def normalize(X):
    mean = np.mean(X, axis=0)
    std = np.std(X, axis=0)
    X_norm = (X - mean) / (std + EPS)
    return X_norm, mean, std

def normalize_apply(X, mean, std):
    """Apply cho VALID/TEST dùng mean/std của TRAIN."""
    return (X - mean) / (std + EPS)

# ===== Load dữ liệu =====
Data_train = pd.read_csv(r"C:\Users\ADMIN\Documents\Tài liệu lưu trữ môn học\Artificial Network\Python Library\Data_Train\merger_vel\merged_train.csv")
Data_test = pd.read_csv(r"C:\Users\ADMIN\Documents\Tài liệu lưu trữ môn học\Artificial Network\Python Library\Data_Train\merger_vel\merged_test.csv")
Valid = pd.read_csv(r"C:\Users\ADMIN\Documents\Tài liệu lưu trữ môn học\Artificial Network\Python Library\Data_Train\merger_vel\valid_vel.csv",)

# ===== Chuyển sang numpy =====
_Data_train = np.array(Data_train)
x_train = _Data_train[:,2:4]
y_train = _Data_train[:,5:]

_Data_test = np.array(Data_test)
x_test = _Data_test[:,2:4]
y_test = _Data_test[:,5:]

_Valid = np.array(Valid)
x_valid = _Valid[:,2:4]
y_valid = _Valid[:,5:]

# ===== Chuẩn hóa dữ liệu =====
x_norm_train, x_mean, x_std = normalize(x_train)
x_norm_test = normalize_apply(x_test, x_mean, x_std)
x_norm_valid = normalize_apply(x_valid, x_mean, x_std)
```

Standardization làm dữ liệu có mean ~ 0 và std ~ 1 à Giúp model học nhanh, ổn định, tránh gradient vẩn đề.

Tạo mô hình

```
# ===== Tạo mô hình =====
Model = NNModel.Model()
Model.add(layer.Layer_Dense_Regularization(2, 8, weights_regularizer_l2=5e-4))
Model.add(activation.Activation_ReLU())
Model.add(layer.Dense_Dropout(0.1))
Model.add(layer.Layer_Dense_Regularization(8, 16, weights_regularizer_l2=5e-4))
Model.add(activation.Activation_ReLU())
Model.add(layer.Dense_Dropout(0.1))
Model.add(layer.Layer_Dense_Regularization(16, 2))
Model.add(activation.Activation_Linear())

Model.set(
    loss=loss_function.Loss_MeanSquaredError(),
    optimizer=optimization.Optimizer_Adam(learning_rate=2e-3, decay=3e-4),
    accuracy=accuracy.Accuracy_Regression()
)
Model.finalize()
```

Mô hình gồm 2 lớp hidden, 2 lớp dropout, 2 lớp activation_ReLU và 1 lớp activation_Linear. Đầu vào input gồm 2 features (error, delta_error), đầu ra output gồm

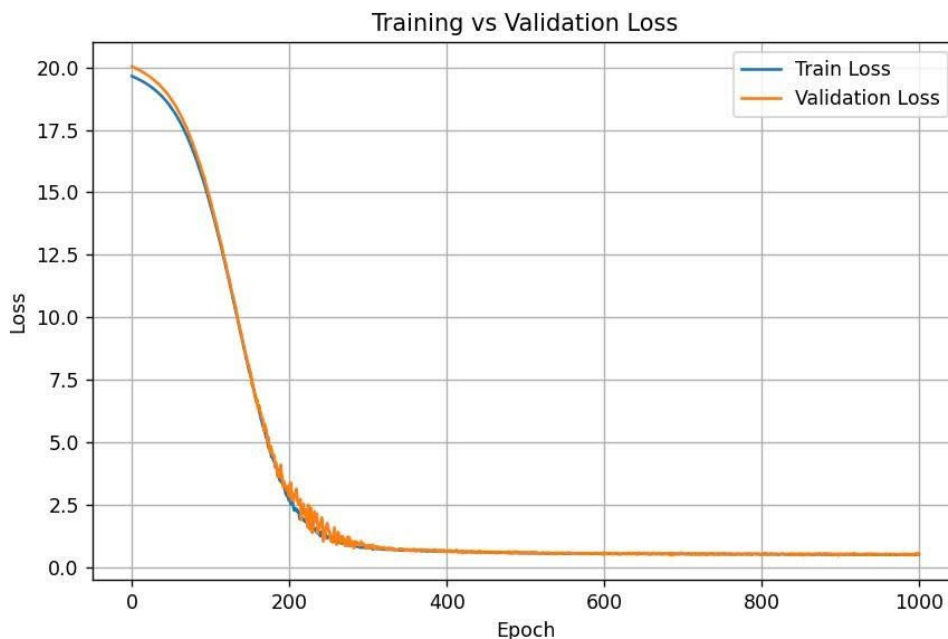
2 features (Kp, Ki), Số neuron lớp hidden thứ nhất là 8 neuron, số neuron lớp hidden thứ 2 là 16 neuron.

Hàm loss dùng là MSE, Optimizer Adam với learning_rate = 0.002

Train mô hình

```
# ===== Train model với validation =====  
history = Model.train(  
    X_norm_train, y_train,  
    validation_data=(X_norm_valid, y_valid),  
    epochs=1000,  
    print_every=10  
)  
  
y_pre = Model.forward(X_norm_test)  
print(y_pre)
```

Số epochs là 1000, có validation. Loss thu được như sau:



Đánh giá hiệu quả mô hình

Hiện tại mô hình chỉ được đánh giá khách quan, chưa có số liệu thuyết phục, chưa đưa ra được cải tiến, so sánh hiệu quả với các mô hình khác.

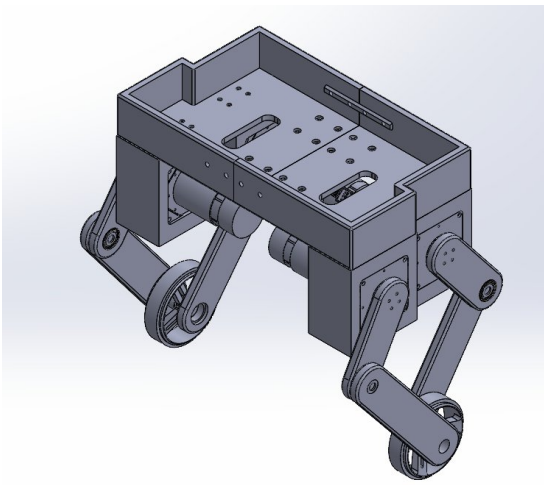
MÔ HÌNH ROBOT BIPEDAL WHEEL DÙNG CƠ CẤU 5-BAR PARALLEL

1. Giới thiệu

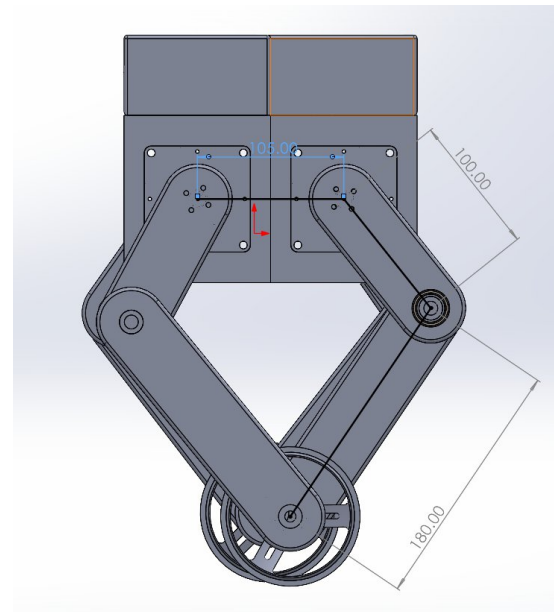
Robot bipedal wheel là hệ robot hai chân có bánh xe ở phía dưới để di chuyển và cân bằng. Trong phiên bản hiện tại, cơ cấu chân đã được rút gọn từ mô hình 3 bậc tự do mỗi bên sang cơ cấu song song 5-bar parallel. Mỗi chân chỉ còn hai khớp chủ động đặt tại A1 và A2, giúp giảm số động cơ, giảm khối lượng, tăng độ cứng vững và đơn giản hóa bài toán điều khiển.

2. Mô hình tổng thể

Robot gồm thân chính ở phía trên, hai cụm chân đối xứng qua mặt phẳng giữa và bánh xe đặt tại điểm cuối của mỗi chân. Mỗi cụm chân sử dụng hai động cơ NF5475 có encoder để điều khiển hai thanh chủ động. Hai thanh bị động tạo thành chuỗi động học kín, từ đó xác định vị trí điểm cuối P của chân robot.



Hình 1. Mô hình 3D tổng thể robot bipedal wheel



Hình 2. Hình chiếu đứng cơ cấu 5-bar parallel

3. Cấu trúc cơ khí 5-bar parallel

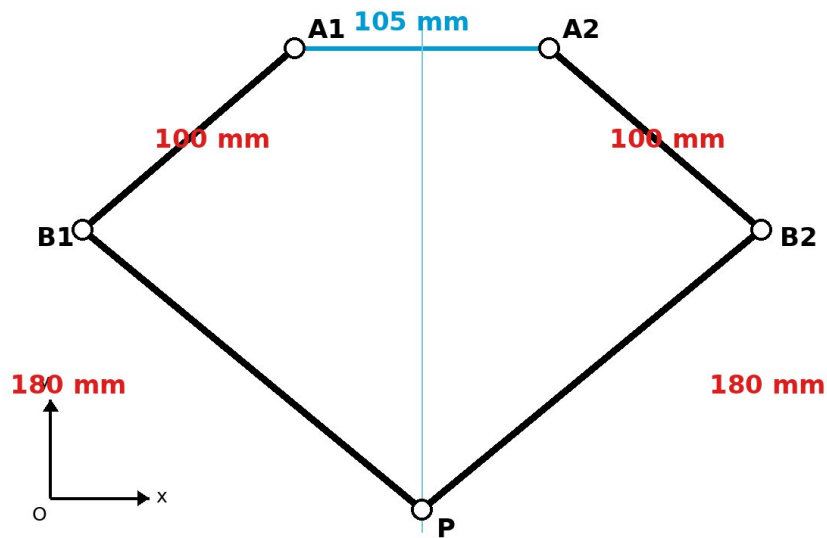
Ký hiệu	Ý nghĩa	Mô tả
A1, A2	Khớp chủ động	Hai khớp quay cố định trên thân, được truyền động bởi động cơ NF5475.
B1, B2	Khớp bị động	Hai khớp quay bị động nối giữa thanh chủ động và thanh bị động.
P	Điểm cuối	Điểm tác dụng của chân robot, gắn với cụm bánh xe hoặc điểm tiếp xúc tương đương.
A1A2	Khoảng cách gốc	Khoảng cách giữa hai khớp chủ động.
Lact	Thanh chủ động	Chiều dài mỗi thanh chủ động A1B1 và A2B2.
Lpas	Thanh bị động	Chiều dài mỗi thanh bị động B1P và B2P.

4. Thông số hình học

Thông số	Ký hiệu	Giá trị
Khoảng cách giữa hai khớp chủ động	A1A2	105 mm
Chiều dài thanh chủ động	Lact	100 mm
Chiều dài thanh bị động	Lpas	180 mm

Ghi chú: Các thông số trên là kích thước thiết kế hiện tại và có thể được hiệu chỉnh trong quá trình thử nghiệm thực tế để phù hợp với kết cấu robot và yêu cầu nhiệm vụ.

5. Sơ đồ cơ cấu 5-bar parallel



Hình 3. Sơ đồ cấu trúc và thông số cơ cấu 5-bar parallel

6. Động học cơ cấu 5-bar parallel

Chọn hệ tọa độ phẳng với gốc tại trung điểm A1A2. Khi đó $A1 = (-d/2, 0)$, $A2 = (d/2, 0)$, trong đó $d = 105$ mm, $L = 100$ mm và $l = 180$ mm. Hai biến khớp chủ động là θ_1 và θ_2 .

6.1. Động học thuận (Forward Kinematics)

Từ θ_1 và θ_2 , trước hết xác định hai điểm B1 và B2:

$$B1 = A1 + L[\cos\theta_1, \sin\theta_1]^T$$

$$B2 = A2 + L[\cos\theta_2, \sin\theta_2]^T$$

Điểm cuối P(x, y) là giao điểm của hai đường tròn tâm B1, B2 và bán kính l. Nghiệm được chọn theo nhánh làm việc thực tế của chân robot.

6.2. Động học nghịch (Inverse Kinematics)

Với vị trí mong muốn Pd(xd, yd), góc chủ động được tính từ bài toán tam giác tại A1 và A2:

$$\theta_1 = \text{atan2}(yd, xd + d/2) \pm \arccos((L^2 + r_1^2 - l^2)/(2Lr_1))$$

$$\theta_2 = \text{atan2}(yd, xd - d/2) \mp \arccos((L^2 + r_2^2 - l^2)/(2Lr_2))$$

trong đó $r_1 = \sqrt{(xd + d/2)^2 + yd^2}$, $r_2 = \sqrt{(xd - d/2)^2 + yd^2}$. Dấu $\pm$ được chọn theo cấu hình lắp thực tế để tránh nhảy nghiệm và tránh vùng kỳ dị.

6.3. Jacobian

Jacobian mô tả quan hệ giữa vận tốc điểm cuối và vận tốc khớp:

$$[\dot{x}, \dot{y}]^T = J(\theta_1, \theta_2)[\dot{\theta}_1, \dot{\theta}_2]^T$$

Ma trận này được dùng để tạo vận tốc đặt cho hai động cơ khi robot cần dịch chuyển điểm cuối P theo quỹ đạo mong muốn. Đồng thời, Jacobian cũng cho biết các vùng làm việc có nguy cơ kỳ dị, nơi sai số nhỏ tại điểm cuối có thể gây vận tốc hoặc mô-men khớp lớn.

7. Vai trò trong hệ thống điều khiển PID kết hợp ANN

- Hai động cơ NF5475 tại A1 và A2 được điều khiển bằng PI để bám tốc độ hoặc góc đặt.
- Từ quỹ đạo mong muốn của điểm cuối P, thuật toán IK/Jacobian sinh ra góc hoặc vận tốc đặt cho từng motor.
- Encoder của motor phản hồi vận tốc/góc thực tế, tạo vòng kín điều khiển bám setpoint.
- ANN được dùng để dự đoán hoặc hiệu chỉnh K_p , K_i theo trạng thái sai số, giúp bộ PI thích nghi tốt hơn khi tải và cấu hình chân thay đổi.
- Cơ cấu 5-bar parallel giúp robot điều chỉnh vị trí chân linh hoạt, hỗ trợ thay đổi trọng tâm và tăng ổn định khi robot cân bằng trên bánh xe.

8. Kết luận phần mô hình robot

Việc chuyển từ mô hình chân 3 bậc tự do sang cơ cấu 5-bar parallel giúp robot giảm độ phức tạp cơ khí và điều khiển nhưng vẫn duy trì khả năng điều chỉnh vị trí điểm cuối. Với kích thước $A_1A_2 = 105$ mm, thanh chủ động 100 mm và thanh bị động 180 mm, cơ cấu tạo được vùng làm việc phù hợp cho chân robot bipedal wheel. Phần điều khiển PI kết hợp ANN trong báo cáo có thể được mở rộng từ điều khiển một động cơ sang điều khiển đồng thời hai động cơ chủ động của cơ cấu 5-bar.